

AI Tutor: An adaptive multi-agent learning system

First Author^{a,*}, Second Author^{b,1} and Third Author^{b,c}

^aShort Affiliation of First Author

^bShort Affiliation of Second Author and Third Author

^cShort Alternate Affiliation of Third Author

ORCID (First Author): <https://orcid.org/.....>, ORCID (Second Author): <https://orcid.org/.....>,
ORCID (Third Author): <https://orcid.org/.....>

Abstract. A platform that provides personalised, comprehensive, and adaptive learning material in a guided manner at a single place is not easily accessible to many learners who wish to study a topic or resolve conceptual doubts. While general-purpose LLMs such as GPT models are widely used for this purpose, they are prone to hallucination and can introduce further confusion rather than clarity. Existing systems lack the guardrails, structured feedback mechanisms, and domain grounding needed to serve as a reliable, end-to-end learning companion. **AI Tutor**, a multi-agent backend system for personalised, adaptive learning. The system is designed in a guided manner with multiple guardrails, rubrics, and critic-discriminator components that verify factual correctness and provide a one-stop solution for learners.

1 Introduction

AI Tutor employs a hierarchical agent architecture in which an Orchestrator Agent coordinates three specialised downstream agents: a Retrieval-Augmented Generation (RAG) Agent that extracts and scores content from uploaded PDF documents, a Teaching Agent that synthesises level-adapted explanations, and a Quiz Agent that generates and evaluates formative assessments. Agents communicate asynchronously via Apache Kafka, and the Orchestrator’s internal workflow is managed by a LangGraph StateGraph with interrupt/resume checkpointing. Our design eliminates heavyweight orchestration frameworks in favour of a thin, deterministic agent loop, reducing operational complexity without sacrificing extensibility..

2 System Architecture

2.1 Overview

AI Tutor is composed of four independent agents running as separate pod. Since GraphState can not be shared for agents deployed under different pods, Inter-agent state shared is mediated exclusively by Apache Kafka, ensuring loose coupling and horizontal scalability. A user session begins when the frontend publishes a `PlannerRequestEvent` to the `init-planner` topic, carrying the user’s natural-language query, optional uploaded PDF paths, and an optional self-declared learner level. The Orchestrator Worker consumes this event and drives the remainder of the session.

* Corresponding Author. Email: somename@university.edu.

¹ Equal contribution.

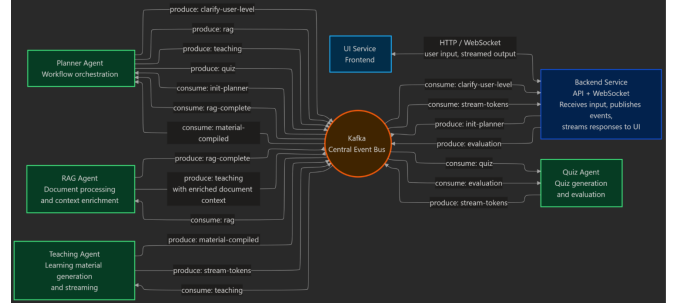


Figure 1. AI Tutor Complete Architecture

2.2 Kafka topic registry

All topic names are centralised in `project/topics.py` through a set of `str`-typed Python Enum classes (e.g. `PlannerTopics`, `PlannerInboundTopics`). No topic name is hard-coded outside this module. Table 1 lists the principal topics.

Topic	Producer	Consumer
<code>init-planner</code>	Frontend	Orchestrator
<code>rag</code>	Orchestrator	RAG Agent
<code>rag-complete</code>	RAG Agent	Orchestrator
<code>teaching</code>	Orchestrator	Teaching Agent
<code>material-compiled</code>	Teaching Agent	Orchestrator
<code>quiz-request</code>	Orchestrator	Quiz Agent
<code>quiz-complete</code>	Quiz Agent	Orchestrator
<code>workflow-complete</code>	Orchestrator	Frontend

Table 1. Principal Kafka topics, producers, and consumers.

Every Kafka message is keyed by `request_id`, co-locating all events of one session on a single partition and preserving strict ordering guarantees.

2.3 Schema contracts

All inter-agent payloads are Pydantic v2 `BaseModel` subclasses defined in `project/schemas.py`. Producers call `.model_dump(mode="json")` before serialisation; consumers re-validate by constructing the model from the raw dict. This approach catches field mismatches at the service boundary rather than silently propagating malformed data into agent logic.

2.4 Orchestrator workflow: LangGraph StateGraph

The Orchestrator implements its session logic as a nine-node LangGraph StateGraph compiled with a MemorySaver checkpoint. The nodes form a directed graph:

The `await_*` nodes call LangGraph's `interrupt()`, suspending the graph and checkpointing the `PlannerState TypedDict` under the session's `request_id`. When the Worker receives a matching Kafka completion event it calls `agent.resume(request_id, payload)`, which delivers the payload via `Command(resume=...)` and continues execution from the exact suspension point—with no polling or external state stores.

All routing between nodes is rule-based. LLM calls occur in exactly two places: (i) level inference when the learner has not self-declared, and (ii) query rewriting when the query is classified as `COMPLEX`.

3 Agent Design

3.1 Orchestrator: level inference and injection gate

Before dispatching any downstream agent, `infer_level` applies a two-stage gate. (1) A rule-based classifier rejects queries containing prompt-injection patterns (e.g. “ignore previous instructions”), producing a `workflow_status = "blocked"` terminal event with no agent dispatch. (2) If the user's level is not pre-declared, a single LLM call resolves both level (`UserLevelEnum`: `beginner`, `intermediate`, `advanced`) and quiz intent, returning a `LevelInferenceResult` with a confidence score. Responses below the confidence threshold route to `clarify_and_end`, which publishes a `ClarifyUserLevelEvent` prompting the user to self-identify. Queries classified as `COMPLEX` trigger a query-rewriting LLM call before any agent is dispatched, improving downstream retrieval quality.

3.2 RAG Retrieval Agent

The RAG Agent consumes `RAGRequestEvent` payloads from the `rag` topic. For each request it receives PDF file paths and a user query.

Page extraction. Text, tables, and images are extracted from each PDF page using `PyMuPDF`. Tables are serialised to Markdown; images are described via a single vision-language model (VLM) call per image.

Relevance scoring. Each page's assembled text is scored against the user query using cosine similarity between sentence embeddings, loaded with `local_files_only=True` for offline inference. Pages scoring below a configurable threshold (default 0.6) are excluded. A bag-of-words fallback activates when the embedding model is unavailable.

Parallel dispatch with bounded concurrency. Pages are dispatched in parallel using a `ThreadPoolExecutor` with a configurable concurrency limit. Deterministic output ordering is preserved by reducing results in pointer order after all workers complete. Failed pages are collected into an error list and excluded from the compilation context (graceful degradation: `status = "partial"` rather than `"failed"`).

Compilation. A single LLM compilation call assembles the retained pages into a topic-focused Markdown study digest, emitted as a `RAGCompletionEvent` on the `rag-complete` topic.

3.3 Teaching Agent

The Teaching Agent consumes `TeachingRequestEvent` from the `teaching` topic, executes an eight-step pipeline, and publishes results to `teaching-complete` (structured response) and `stream-tokens` (live token events). A completion event is always published — even on error.

Three-Mode Explanation Pipeline. Routes to `beginner` (diagram required), `intermediate`, or `advanced` prompts based on `user_level`; unrecognised mode errors before any LLM call.

Structured Streaming. Output is parsed by a streaming field extractor: *Explanation*, *Notes*, *Example* stream token-by-token; *Diagram* is buffered whole. A `done-sentinel` always closes the stream.

Guardrail Classification. A fast LLM call classifies first-turn messages into four categories: `greeting`, `off-topic`, `unclear`, or `valid`; non-learning inputs return a canned response and skip the pipeline. Bypassed on follow-up turns; fails open.

Reflection Loop (POC, disabled). Runs *N* critique-revision cycles post-generation; disabled by default due to time constraints.

Multi-Turn Conversation. `chat_history` (oldest-to-newest) is prepended to LLM messages; `ConversationStore` auto-populates per `Socket.IO sid` with a configurable cap and 300s TTL.

RAG Context Integration. `context` injects RAG-compiled material into all mode prompts (primary source); truncated to 4000 chars. Falls back to general knowledge when empty.

Limitations of the current implementation and planned future enhancements are discussed in [Appendix A.1](#).

3.4 Quiz-Evaluation Agent

When quiz intent is detected, the Orchestrator publishes a `QuizRequestEvent` carrying the user prompt, all learner levels, and the full `teaching_materials` map. The Quiz Agent generates a `Quiz` containing multiple-choice (single and multi-select) and descriptive questions calibrated to each level, annotated with per-option explanations and rubrics. A separate evaluation phase accepts learner answers and returns a `QuizResult` with per-question scoring, weak sub-concept detection, and a recommended next action (`"re-teach"`, `"practice-more"`, or `"advance"`).

Acknowledgements

By using the `ack` environment to insert your (optional) acknowledgements, you can ensure that the text is suppressed whenever you use the `doubleblind` option. In the final version, acknowledgements may be included on the extra page intended for references.

A Teaching Agent

A.1 Limitations and Future Scope

A.1.1 Current Limitations

- L1. Structural-only Mermaid validation.** `validate_mermaid()` checks for a recognised header and at least one content segment. A diagram that is structurally valid but semantically broken (e.g. malformed edge syntax that a renderer would reject) can pass this check. A full Mermaid parser is deferred to a future iteration.
- L2. Process-local conversation store.** `ConversationStore` lives in memory inside the worker process. A process restart clears all in-flight session histories. This is an accepted trade-off for a short-session tutoring product (sessions expire after 5 minutes of idle time), but would need to be replaced with a persistent store (Redis, database) for longer-lived sessions.
- L3. No cross-worker history sharing.** If the Teaching Agent is scaled horizontally, each worker process owns an independent `ConversationStore`. Requests from the same `sid` reaching different workers will not see each other's history. Sticky routing or an external shared store would be required for multi-process deployments.
- L4. Reflection quality is probabilistic.** The critique-revision loop is expected to improve output quality in the majority of runs, but it is a probabilistic guarantee. A cycle may occasionally produce a lateral move rather than an improvement. The automated test suite verifies control flow and token accounting; output quality validation requires real LLM calls and manual review.

A.1.2 Future Enhancements

- F1. Full Mermaid parsing.** Replace regex-based structural validation with a proper Mermaid parser to catch semantic errors before they reach the frontend renderer.
- F2. Persistent session storage.** Migrate `ConversationStore` to a Redis-backed or database-backed store to support multi-process deployments and survive worker restarts.
- F3. Adaptive mode selection.** Allow the agent (or the Planner) to adjust the `user_level` dynamically based on learner performance signals from the Evaluation Agent, personalising the explanation depth over time.
- F4. Richer output formats.** Extend the four-section schema with optional sections such as *Quiz Preview* or *Related Topics* to provide tighter integration with downstream agents without breaking the existing contract.
- F5. Streaming reflection.** Stream revised content from the reflection loop back to the frontend in real time rather than waiting for all critique-revision cycles to complete before emitting the final output.